guarantee writing to it. This is why *let it crash* was born in the rack-and-stack world of the telcos, where failed machines were common enough to make their availability goals impossible without a plan that accounted for them.

Because you can't prevent all failures from happening, you have to adopt a strategy that keeps the following in mind:

- Things break. The system needs to be *fault tolerant* so it can stay available and continue to run. Recoverable faults shouldn't trigger catastrophic failures.
- In some cases, it's acceptable if the most important features of the system stay available as long as possible while failing parts are stopped and cut off from the system so they can't interfere with the rest of the system and produce unpredictable results.
- In other cases, certain components are so important that they need active backups (probably on a different server or using different resources) that can kick in when the main component fails to quickly remedy the unavailability.
- A failure in certain parts of the system shouldn't crash the entire system, so you need a way to isolate particular failures that you can deal with later.

Of course, the Akka toolkit doesn't include a fault-tolerance silver bullet. You still have to handle specific failures, but you can do so in a cleaner, more application-specific way. The Akka features in table 5.1 enable you to build the fault-tolerant behavior you need.

Table 5.1 Fault-avoidance strategies